DAA Assignment 2

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 2 – CO2

CO2 Coverage: Brute-Force and Exhaustive-Search techniques – Selection Sort, Bubble Sort, Insertion Sort, Sequential Search, Brute-Force String Matching, Closest Pair, Convex Hull and Exhaustive Search.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Q1 – Sorting Under Different Data Conditions **6 Marks**

Problem Context – E-Commerce Order Processing

An e-commerce fulfilment centre arranges customer orders according to a **dispatch priority score**.

The development team is considering:

Selection Sort | Bubble Sort | Insertion Sort

However, the incoming data may be randomly ordered, nearly sorted, already sorted or significantly disordered.

Your Challenge

Investigate whether one of the three algorithms should always be preferred or whether the appropriate choice depends on the **characteristics of the input data**.

Before implementation:

- Predict how the three algorithms will behave under different input arrangements.
- Explain the algorithmic reason for your prediction.

Design **two deliberately contrasting datasets** and **one additional edge-case dataset of your own** that will expose meaningful behavioural differences among the algorithms.

Implement the three algorithms and record appropriate measures such as:

- Number of comparisons
- Number of swaps/movements
- Execution time, where useful

Compare your prediction with the observed behaviour.

Design Decision

The fulfilment centre reports that approximately **95% of existing orders are already in the correct priority order**, with only small batches of new orders added periodically.

Determine which of the three algorithms you would recommend and defend your decision.

Evidence Expected

Reasoned prediction, three deliberately designed input conditions, implementation of the three sorting algorithms, operation counts, comparison of predicted and observed behaviour, and an evidence-based recommendation.

Q2 – When Is Sequential Search an Appropriate Engineering Choice? 4 Marks

Problem Context – Aircraft Maintenance Tool Tracking

An aircraft maintenance facility maintains a temporary list of specialised tools currently issued to technicians.

Each record contains:

Tool ID | Technician ID | Issue Time | Work Bay

The list changes continuously as tools are issued and returned.

When a technician asks:

> "Who currently has Tool T-184?"

the system uses **Sequential Search**.

Your Challenge

Design experiments to investigate Sequential Search under:

- Successful searches at different positions

- Unsuccessful searches
- Different list sizes

Record the **number of comparisons** required.

Then answer:

> **Under what operating conditions is Sequential Search a reasonable engineering choice, and when would you stop recommending it?**

Consider both:

- Dataset size
- Frequency of search requests

Changed Condition

The organisation expands from one hangar to **20 maintenance facilities**, resulting in substantially more active records and lookup requests.

Re-evaluate your original recommendation.

Evidence Expected

Designed search experiments, comparison-count evidence, analysis of successful and unsuccessful searches, an evidence-based operating boundary and a justified response to the changed operating condition.

Q3 – Engineer the Behaviour of Brute-Force String Matching 6 Marks

Problem Context – Cybersecurity Log Screening

A security operations centre searches large text-based system logs for known event signatures represented as character patterns.

The initial system uses **Brute-Force String Matching**.

Your Challenge

Instead of merely implementing the algorithm, investigate **how the structure of the text and pattern influences computational effort**.

Design:

1. One text-pattern pair in which the algorithm performs relatively little work.

2. One text-pattern pair deliberately engineered to cause a **large number of character comparisons**.
3. One additional test condition of your own choice.

Before executing each test:

- Predict the expected behaviour.
- Explain the algorithmic reason for your prediction.

Implement the algorithm and record the number of character comparisons.

Inquiry

Compare your prediction with the experimental result.

Explain:

- Why one input requires substantially more work than another
- Which characteristics of the text and pattern create repeated comparisons
- What this reveals about evaluating an algorithm using only its input size

Evidence Expected

Correct implementation, three deliberately designed test conditions, prediction before execution, character-comparison evidence, and a reasoned explanation of the observed differences.

Q4 – Closest Pair and the Cost of Examining Every Possibility 6 Marks

Problem Context – Drone Collision-Risk Monitoring

A logistics company operates delivery drones. At a particular instant, each drone reports its position:

[
D_i=(x_i,y_i)
]

The monitoring system must identify the **two drones closest to one another**.

Your Challenge

Design a **Brute-Force Closest-Pair** solution.

Before implementation, determine:

- Which candidate pairs must be examined
- How duplicate pair comparisons can be avoided
- How many unique pairs exist for (n) drones

Derive the number of candidate pairs mathematically.

Implement your solution and test it using **at least three meaningfully different input sizes**.

Record:

- Number of pairs examined
- Number of distance calculations
- Execution time, where useful

Compare the theoretical growth with experimental behaviour.

Scalability Inquiry

Define what you consider an **acceptable response time** for your experimental environment.

Using your evidence, determine approximately when the brute-force approach begins to become impractical.

Finally, identify **where the computational inefficiency originates** and describe what a redesigned strategy would need to avoid or reduce.

You are not required to implement an advanced Closest-Pair algorithm in this question.

Evidence Expected

Algorithm design, mathematical pair-count derivation, implementation, experimental growth evidence, a defined acceptability criterion and reasoned identification of the limitation of the brute-force strategy.

Q5 – Derive a Convex Hull Algorithm from a Geometric Principle 7 Marks

Problem Context – Wildlife Reserve Boundary Monitoring

Monitoring stations are positioned throughout a wildlife reserve. Each station is represented by coordinates:

[

$P_i=(x_i,y_i)$
]

Conservation officers need to determine the **outer boundary formed by the monitoring stations**.

You are required to develop a **Brute-Force Convex Hull** solution.

Design Starting Point

You are given only the following geometric principle:

> **A line joining two candidate points may form part of the outer boundary if all remaining points lie on an appropriate single side of that line.**

You are **not provided with the complete algorithm**.

Your Challenge

Convert this geometric property into a computational procedure.

Determine:

- How two candidate points define a line
- How the position of another point relative to that line can be determined
- How the remaining points must be examined
- What condition identifies a potential hull edge
- How the final boundary will be constructed

Develop your own pseudocode before implementation.

Test Your Design

Test the algorithm using:

- A point set containing both boundary and interior points
- A second point configuration deliberately designed by you to challenge your algorithm

Visualise the resulting boundary.

Analyse how the computational work grows as the number of points increases.

Correctness Inquiry

Explain:

Why does your geometric test provide evidence that an accepted edge belongs to the outer boundary?

Your explanation should be based on geometric reasoning rather than simply referring to the program output.

Evidence Expected

A student-derived geometric decision rule, pseudocode, implementation, deliberately designed test configurations, computational analysis, boundary visualisation and a reasoned correctness argument.

Q6 – Exhaustive Search and Combinatorial Growth **8 Marks**

Problem Context – Humanitarian Airlift Planning

A helicopter carrying emergency supplies has a strict payload limit.

Each available package contains:

Package ID | Weight | Humanitarian Value

The objective is to select a combination of packages that:

- Does not exceed the payload limit, and
- Maximises the total humanitarian value delivered.

Your Challenge

Design an **Exhaustive-Search strategy**.

Determine:

- How candidate package combinations will be represented
- How candidate solutions will be generated
- How feasibility will be checked
- How feasible solutions will be evaluated
- How the best solution will be retained

Do not begin from supplied pseudocode.

Correctness Check

Create a small instance for which the possible combinations can be independently verified.

Use this to establish whether your algorithm identifies the correct optimum.

Growth Investigation

Investigate increasing problem sizes.

Before experimentation:

- Derive/predict how the number of candidate combinations grows with (n).
- Predict what will happen as additional packages are introduced.

Use **at least three meaningful problem sizes** to validate your prediction experimentally.

Inquiry

Explain:

> **Why can adding what appears to be a relatively small number of additional packages produce a dramatic increase in computational effort?**

Your explanation must relate to the growth of the **solution space**.

Deployment Decision

Assume that the airlift planner must produce a loading decision within a fixed response time selected and justified by you.

Using your experimental evidence, estimate the largest problem size for which you would be willing to deploy your exhaustive-search solution.

Evidence Expected

Student-designed exhaustive enumeration, correctness verification, mathematical prediction of solution-space growth, experimental validation, explanation of combinatorial growth and an evidence-based deployment boundary.

DAY-4 DESIGN CHALLENGE

Q7 – Diagnose, Redesign and Validate 13 Marks

Review the evidence you generated while working on Q1–Q6.

Select **ONE solution** that demonstrates a significant computational limitation as its problem size or operating requirement increases.

You are **not permitted simply to replace the solution with a more advanced textbook algorithm and state that its complexity is better**.

Your task is to understand the computational problem first and then develop an appropriate redesign.

A. Diagnose the Computational Bottleneck — 2 Marks

Using your own mathematical and/or experimental evidence, identify:

- Where most computational effort is being spent
- Which operation grows significantly as the input increases
- Whether work is being unnecessarily repeated
- Which characteristic of the original algorithm causes the limitation

Evidence Expected

A precise diagnosis supported by evidence from your original implementation and analysis.

B. Define the Design Objective — 2 Marks

Before proposing a new solution, clearly state what your redesign must achieve.

For example:

- Reduce unnecessary comparisons
- Avoid examining every candidate
- Exploit existing ordering
- Reuse previously computed information
- Reduce the number of candidate solutions
- Reduce repeated processing

Specify a **measurable objective** wherever possible.

Evidence Expected

A clearly stated redesign objective derived directly from the identified computational limitation.

C. Design the Revised Computational Strategy **4 Marks**

Develop a revised solution.

Explain:

- What you have changed
- What information or property your new design exploits
- Which operations are avoided or reduced
- Whether preprocessing is introduced
- Whether additional memory is required
- Any new trade-offs created by the redesign

Provide appropriate pseudocode or a design representation.

Your marks will be based on the **quality of the algorithmic reasoning**, not merely on whether you name a known algorithm.

Evidence Expected

A coherent redesigned computational strategy directly addressing the diagnosed bottleneck, supported by appropriate design representation and technical justification.

D. Predict Before Implementing **1 Mark**

Before testing the redesigned solution:

- Predict how its behaviour should differ from the original.
- Identify the performance measure in which you expect improvement.

Evidence Expected

A clear, testable prediction made before experimental validation.

E. Implement and Validate the Redesign **2 Marks**

Implement a prototype of the redesigned strategy wherever feasible.

Using the **same or equivalent test conditions**, compare:

Original Solution → Redesigned Solution

Use appropriate measures such as:

- Comparisons
- Key operations
- Candidate solutions examined
- Distance calculations
- Character comparisons
- Execution time, where relevant

Evidence Expected

Comparable experimental evidence showing the behaviour of the original and redesigned solutions.

F. Defend the Final Design **2 Marks**

Evaluate whether the redesign achieved its intended objective.

Discuss:

- Whether the prediction was supported
- Magnitude of improvement, if any
- New costs or trade-offs
- Situations in which the original solution may still be preferable
- Whether the redesigned solution sufficiently addresses the identified limitation

A redesign that does **not** outperform the original will not automatically be considered unsuccessful if you correctly analyse and explain why the design hypothesis did not produce the expected result.

Evidence Expected

A critical, evidence-based evaluation of the redesigned solution and a defensible final engineering conclusion.

MARK DISTRIBUTION

Question	Assessed Area	Marks
Q1	Selection, Bubble & Insertion Sort	**6**
Q2	Sequential Search	**4**

Q3	Brute-Force String Matching	**6**
Q4	Brute-Force Closest Pair	**6**
Q5	Brute-Force Convex Hull	**7**
Q6	Exhaustive Search	**8**
Q7	**Day-4 Design Challenge – Diagnose, Redesign & Validate**	**13**
	TOTAL	**50**

Submission Requirements

Submit:

- Your assignment report containing the required evidence for Q1–Q7
- Source code for all implemented solutions
- Raw experimental data used to produce your results
- Tables/graphs used in your analysis, where applicable

Operation counts, comparison counts or other algorithm-specific measures should be treated as **primary experimental evidence**. Execution time may be reported as additional evidence but should not be used as the sole basis for your conclusions.

Assessment Verification

Students may be asked to explain any part of their:

- Algorithmic design
- Source code
- Test-data design
- Experimental results
- Predictions
- Analysis
- Design Challenge

during a brief verification discussion/viva.

The student should therefore be able to **explain and defend the evidence submitted as their own work**.

Overall Assessment Expectation

This assignment does not primarily assess your ability to reproduce textbook algorithms.

It assesses your ability to:

Predict → Design → Implement → Measure → Analyse → Evaluate → Diagnose → Redesign → Validate

Your conclusions and design decisions must be supported by **your own mathematical reasoning and experimental evidence**.

Q1 – Sorting Under Different Data Conditions

1.1 Problem and objective

An e-commerce fulfilment centre sorts orders by dispatch-priority score. Selection Sort, Bubble Sort, and Insertion Sort are compared on already-sorted, reverse-sorted, and nearly-sorted lists. The objective is to decide whether one algorithm is always best or whether the data arrangement should influence the choice.

1.2 Predictions before implementation

Input condition	Selection Sort prediction	Bubble Sort prediction	Insertion Sort prediction
Already sorted	Still makes about $n(n-1)/2$ comparisons; swaps are few.	With an early-stop flag, one pass detects no swaps; about $n-1$ comparisons.	About $n-1$ comparisons and no shifts; very efficient.
Reverse sorted	Quadratic comparisons and up to $n-1$ swaps.	Quadratic comparisons and many swaps.	Quadratic comparisons and many shifts.
Nearly sorted	Comparison count remains quadratic; movement is limited.	Early-stop can help if only a few inversions exist.	Usually close to linear because only a few elements move.

Reasoning: Selection Sort repeatedly scans the unsorted suffix to find a minimum, regardless of existing order. Optimised Bubble Sort can stop when a complete pass has no swaps. Insertion Sort inserts each item into the already-processed prefix, so its work is proportional to the number of inversions as well as the list length.

1.3 Deliberately designed datasets

Dataset	Values	Purpose
A – Already sorted	[1, 2, 3, 4, 5, 6, 7, 8]	Tests whether an algorithm benefits from existing order.
B – Reverse sorted	[8, 7, 6, 5, 4, 3, 2, 1]	Creates many inversions and exposes movement cost.
C – Nearly sorted	[1, 2, 3, 5, 4, 6, 7, 8]	Contains one local disorder to test adaptive behaviour.

1.4 Python implementation with operation counts

```
def selection_sort(values):
    a = values[:]
    comparisons = swaps = 0
    n = len(a)
    for i in range(n - 1):
        min_i = i
        for j in range(i + 1, n):
            comparisons += 1
            if a[j] < a[min_i]:
                min_i = j
        if min_i != i:
            a[i], a[min_i] = a[min_i], a[i]
            swaps += 1
    return a, comparisons, swaps


def bubble_sort(values):
```

```
    a = values[:]
    comparisons = swaps = 0
    n = len(a)
    for end in range(n - 1, 0, -1):
        changed = False
        for j in range(end):
            comparisons += 1
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swaps += 1
                changed = True
        if not changed:
            break
    return a, comparisons, swaps


def insertion_sort(values):
    a = values[:]
    comparisons = shifts = 0
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0:
            comparisons += 1
            if a[j] > key:
                a[j + 1] = a[j]
                shifts += 1
                j -= 1
            else:
```

```
            break
        a[j + 1] = key
    return a, comparisons, shifts


datasets = {
    "Already sorted": [1,2,3,4,5,6,7,8],
    "Reverse sorted": [8,7,6,5,4,3,2,1],
    "Nearly sorted": [1,2,3,5,4,6,7,8]
}
for name, data in datasets.items():
    print("\n", name, data)
    print("Selection:", selection_sort(data))
    print("Bubble:   ", bubble_sort(data))
    print("Insertion:", insertion_sort(data))
```

1.5 Operation-count evidence

Dataset	Selection Sort	Optimised Bubble Sort	Insertion Sort
Already sorted	28 comparisons, 0 swaps	7 comparisons, 0 swaps	7 comparisons, 0 shifts
Reverse sorted	28 comparisons, 4 swaps	28 comparisons, 28 swaps	28 comparisons, 28 shifts
Nearly sorted	28 comparisons, 1 swaps	13 comparisons, 1 swaps	8 comparisons, 1 shifts

The values are operation counts produced by the counting rules in the code above. Selection Sort performs exactly $n(n-1)/2$ key comparisons for every input of length n. Bubble Sort counts adjacent comparisons and stops after a no-swap pass. Insertion Sort counts each key comparison inside its while-loop and each right-shift.

1.6 Interpretation and recommendation

The prediction is supported: Insertion Sort and early-stop Bubble Sort exploit existing order, while Selection Sort does not reduce its comparison count. For a workload where approximately 95% of orders are already correctly ordered and only small batches are added, Insertion Sort is recommended for small incremental updates. It needs little extra memory and performs close to linear work when the list is nearly sorted. If the data must be repeatedly sorted in large batches, a production system should also consider a more scalable library sort; however, among the three required algorithms, Insertion Sort best matches the stated input condition.

Limitation: the conclusion depends on small batches and near-sorted data. If the order becomes random or reverse-sorted, Insertion Sort can require quadratic movement. The recommendation should be revisited if the workload changes.

Q2 – When Is Sequential Search an Appropriate Engineering Choice?

2.1 Problem and method

The aircraft maintenance tool list is temporary and changes whenever tools are issued or returned. Sequential Search checks records from the beginning until the requested Tool ID is found or the list ends. For a successful search at position k (counting from 1), k comparisons are required. An unsuccessful search in a list of n distinct IDs requires n comparisons.

2.2 Experiment design

Experiment	List size	Target position/result	Expected comparisons
A	8	First record	1
B	8	Middle record (position 4)	4
C	8	Last record (position 8)	8
D	8	Not present	8
E	100	Last record	100
F	1,000	Not present	1,000

2.3 Python implementation

```
def sequential_search(records, target):
    comparisons = 0
    for index, record in enumerate(records):
        comparisons += 1
        if record == target:
            return index, comparisons
```

```
    return -1, comparisons


tools = [f"T-{i:03d}" for i in range(1, 9)]
tests = [("T-001", "first"), ("T-004", "middle"),
         ("T-008", "last"), ("T-999", "missing")]
for target, label in tests:
    position, count = sequential_search(tools, target)
    print(label, "target =", target,
          "position =", position, "comparisons =", count)
```

2.4 Results and analysis

For a list of eight tools, the expected counts are 1, 4, 8, and 8 for first, middle, last, and missing targets respectively. The best case is O(1), while the worst case is O(n). If successful targets are equally likely to appear at any position, the average successful comparison count is (n+1)/2. A missing target always requires n comparisons.

List size n	First item	Middle item	Last item	Missing item
8	1	4	8	8
100	1	50	100	100
1,000	1	500	1,000	1,000

2.5 Operating boundary and changed condition

Sequential Search is reasonable when the list is small, search requests are infrequent, the list changes frequently, or maintaining a separate index would cost more than scanning. It is simple and requires no preprocessing or extra index memory.

It becomes a poor choice when the record count and request frequency are both high. If a list of 100,000 records receives 10,000 unsuccessful requests, the worst-case workload can reach about 1,000,000,000 record comparisons. Expanding from one hangar to 20 facilities increases the active data and lookup demand, so the system should use a hash

table keyed by Tool ID or a database index. These structures require extra memory and update management, but they avoid scanning every record for each query.

Engineering recommendation: keep Sequential Search for a small temporary list, with a review threshold based on measured request volume. For multi-facility operation, use indexed lookup and retain a sequential scan only for small local lists or diagnostic fallback.

Q3 – Engineer the Behaviour of Brute-Force String Matching

3.1 Algorithm and prediction

Brute-Force String Matching aligns a pattern of length m at every possible text position and compares characters from left to right. If the text has length n, there are at most $n-m+1$ alignments. The worst-case number of character comparisons is $O((n-m+1)m)$, which is $O(nm)$.

Test	Text	Pattern	Prediction
A – Quick match	SECURITY ALERT FOUND	ALERT	Relatively little work because the pattern is found after only a few alignments.
B – Repeated-prefix worst case	AAAAAAAAAAAAB	AAAAB	Many alignments compare several A characters before failing; one final alignment may match.
C – No match	NETWORK LOG CLOSED	XYZ	Most alignments fail on the first character, so the count is comparatively low.

3.2 Python implementation

```
def brute_force_match(text, pattern):
    comparisons = 0
    n, m = len(text), len(pattern)
    if m == 0:
        return 0, 0
    for start in range(n - m + 1):
        j = 0
        while j < m:
            comparisons += 1
            if text[start + j] != pattern[j]:
                break
            j += 1
        if j == m:
            return start, comparisons
    return -1, comparisons


tests = [
    ("SECURITY ALERT FOUND", "ALERT"),
    ("AAAAAAAAAAAAAB", "AAAAB"),
    ("NETWORK LOG CLOSED", "XYZ")
]
for text, pattern in tests:
    index, count = brute_force_match(text, pattern)
    print("Text:", text)
    print("Pattern:", pattern, "index:", index,
          "character comparisons:", count)
```

3.3 How repeated comparisons arise

At each alignment, the algorithm begins from the pattern's first character. When text and pattern share a long prefix, the algorithm repeats comparisons of that prefix at nearby positions before discovering a mismatch. A text made of repeated A characters and a pattern such as AAAAB is therefore a useful stress test: many alignments match several A characters and then fail at B.

By contrast, if the first pattern character differs from most text characters, each alignment can fail after one comparison. A pattern that occurs early also allows the algorithm to stop before checking all alignments. Consequently, text length alone does not explain the actual operation count; pattern length, prefix structure, mismatch position, and match location also matter.

3.4 Experimental evidence table

Text	**Pattern**	**Match index (−1 = absent)**	**Character comparisons**
SECURITY ALERT FOUND	ALERT	9	14
AAAAAAAAAAAAB	AAAAB	9	50
NETWORK LOG CLOSED	XYZ	-1	16

The comparison counts in the table are reproducible from the included function. The second case is deliberately designed to repeat prefix work; its count should be interpreted alongside the exact chosen text and pattern rather than as a universal count for every repeated-character input.

3.5 Conclusion

Brute-force matching is easy to implement and useful for short strings, small logs, or a one-time scan. For large logs and repeated queries, its repeated prefix comparisons can be expensive. A redesign should reuse information about previously matched prefixes or use a suitable search index. This question demonstrates why input structure and operation counts are essential when evaluating an algorithm.

Q4 – Closest Pair and the Cost of Examining Every Possibility

4.1 Candidate pairs and mathematical derivation

For n drone positions, every distinct pair must be considered by the brute-force approach. To avoid duplicates, select the first drone i and pair it only with drones j greater than i. This examines (0,1), (0,2), …, but never (1,0), and never compares a drone with itself.

There are n choices for the first endpoint and n−1 for the second if order is counted. Every unordered pair appears twice in that count, so the number of unique pairs is $n(n-1)/2 = C(n,2)$.

4.2 Python implementation

```
from math import sqrt
from time import perf_counter


def closest_pair(points):
    if len(points) < 2:
        raise ValueError("At least two points are required")
    best_pair = None
    best_sq = float("inf")
    pairs_examined = 0
    distance_calculations = 0
    for i in range(len(points) - 1):
        for j in range(i + 1, len(points)):
            pairs_examined += 1
            dx = points[i][0] - points[j][0]
            dy = points[i][1] - points[j][1]
            distance_sq = dx * dx + dy * dy
            distance_calculations += 1
```

```
            if distance_sq < best_sq:
                best_sq = distance_sq
                best_pair = (points[i], points[j])
    return best_pair, sqrt(best_sq), pairs_examined, distance_calculations


samples = {
    10: [(i * 7 % 19, i * 11 % 23) for i in range(10)],
    100: [(i * 37 % 211, i * 53 % 223) for i in range(100)],
    500: [(i * 37 % 1009, i * 53 % 1013) for i in range(500)]
}
for n, pts in samples.items():
    start = perf_counter()
    result = closest_pair(pts)
    elapsed = perf_counter() - start
    print(n, "pairs =", result[2],
          "distance calculations =", result[3],
          "time (seconds) =", round(elapsed, 6))
```

4.3 Theoretical and experimental growth

Number of drones n	Unique pairs n(n−1)/2	Distance calculations
10	45	45
100	4,950	4,950
500	124,750	124,750
1,000	499,500	499,500
10,000	49,995,000	49,995,000

The number of distance calculations equals the number of unique pairs. Doubling n approximately quadruples the work because n(n−1)/2 is quadratic. For large n, the dominant term is $n^2/2$.

4.4 Response-time criterion and scalability

For this prototype, define an acceptable response time as no more than 1 second for a single update on the test computer. This is a stated engineering target, not a universal standard. Run the supplied timing loop locally to identify the largest n that meets it, because hardware and interpreter performance differ.

A rough estimate can be made from a measured test. If 500 points require T seconds, then a first-order estimate for N points is $T \times N(N-1)/(500\times499)$, because the operation count scales with the pair count. This estimate must be validated with another run.

The inefficiency comes from evaluating every pair, including many pairs that cannot plausibly be the closest once spatial structure is known. A redesigned strategy should avoid or prune candidate pairs using spatial ordering, divide-and-conquer, or a spatial index. Such strategies add implementation complexity and may require sorting or extra memory.

4.5 Conclusion

Brute force is appropriate for small point sets and for validating a more advanced implementation. Its simple pair enumeration is easy to verify, but quadratic growth makes it unsuitable for very large drone fleets under a strict response-time requirement.

Q5 – Derive a Convex Hull Algorithm from a Geometric Principle

5.1 Geometric decision rule

For candidate points A=(x1,y1) and B=(x2,y2), determine the side of the directed line AB on which a point P=(x,y) lies using the cross product:

cross(A, B, P) = (x2−x1)(y−y1) − (y2−y1)(x−x1).

- A positive value means P lies on one side of directed line AB.
- A negative value means P lies on the opposite side.
- Zero means the three points are collinear.

If every other point lies on one side of the line AB, then AB is a supporting line of the point set and can be a convex-hull boundary edge. If all points are collinear, the endpoints define the hull segment; intermediate points are not needed as corner vertices.

5.2 Student-designed pseudocode

```
BRUTE_FORCE_HULL(points):
    remove duplicate points
    if fewer than 3 distinct points:
        return the points in boundary order

    edges = empty set
    for each unordered pair A, B:
        positive = false
        negative = false
        for each point P other than A and B:
            side = cross(A, B, P)
            if side > 0: positive = true
            if side < 0: negative = true
        if not (positive and negative):
            add supporting edge A-B
```

```
    collect endpoints of supporting edges
    sort hull vertices by angle around their centroid
    return sorted vertices
```

For collinear boundary points, the implementation below collects hull vertices and sorts them around the centroid. A production-quality implementation should explicitly handle all-collinear input and collinear points along edges.

5.3 Python implementation

```
from math import atan2

def cross(a, b, p):
    return ((b[0] - a[0]) * (p[1] - a[1])
            - (b[1] - a[1]) * (p[0] - a[0]))

def brute_force_hull(points):
    pts = list(dict.fromkeys(points))
    if len(pts) <= 2:
        return pts
    edge_vertices = set()
    for i in range(len(pts)):
        for j in range(i + 1, len(pts)):
            positive = negative = False
            for k, p in enumerate(pts):
                if k == i or k == j:
                    continue
                value = cross(pts[i], pts[j], p)
                if value > 0: positive = True
                elif value < 0: negative = True
```

```
            if not (positive and negative):
                edge_vertices.add(pts[i])
                edge_vertices.add(pts[j])
    hull = list(edge_vertices)
    cx = sum(p[0] for p in hull) / len(hull)
    cy = sum(p[1] for p in hull) / len(hull)
    hull.sort(key=lambda p: atan2(p[1]-cy, p[0]-cx))
    return hull

set_a = [(0,0),(4,0),(4,3),(0,3),(2,1),(1,2),(3,1)]
set_b = [(0,0),(2,0),(4,0),(4,4),(2,2),(0,4),(2,4)]
print("Set A hull:", brute_force_hull(set_a))
print("Set B hull:", brute_force_hull(set_b))
```

5.4 Test configurations and expected boundary

Test	Points	Expected outer boundary
A – Interior points	(0,0), (4,0), (4,3), (0,3), (2,1), (1,2), (3,1)	(0,0), (4,0), (4,3), (0,3)
B – Collinear edge points	(0,0), (2,0), (4,0), (4,4), (2,2), (0,4), (2,4)	Corners: (0,0), (4,0), (4,4), (0,4); (2,2) is interior.

Visualisation guidance: plot all input points as scatter markers, then draw a closed line through the returned hull vertices. The returned vertices should form the exterior polygon; interior points should remain inside it.

5.5 Correctness reasoning

A convex polygon's boundary edges lie on supporting lines: all points in the set are on the polygon's interior side or on the line itself. Therefore, if all input points lie on one side of the line through A and B, that line supports the point set and A–B is a candidate

boundary edge. If points occur strictly on both sides, the line cuts through the point set and cannot be a supporting boundary line.

5.6 Complexity and limitations

There are $O(n^2)$ candidate point pairs. For each pair, the algorithm checks up to n points, so the side-testing phase takes $O(n^3)$ time. Extra storage for points and edge endpoints is typically O(n). The method is clear and useful for small point sets, but cubic growth makes it costly for dense monitoring networks. A hardened implementation should explicitly handle all-collinear points and edge-collinear points.

Q6 – Exhaustive Search and Combinatorial Growth

6.1 Candidate representation and feasibility

Represent each package combination by a binary choice vector: 1 means selected and 0 means not selected. For n packages, there are 2^n candidate subsets, including the empty subset.

- Generate each subset using a bit mask from 0 through 2^n-1.
- Sum the selected package weights and reject a subset if it exceeds capacity.
- For feasible subsets, sum humanitarian values.
- Keep the feasible subset with the greatest total value.

6.2 Python implementation

```
def exhaustive_airlift(packages, capacity):
    n = len(packages)
    best_value = 0
    best_weight = 0
    best_ids = []
    candidates = feasible = 0
    for mask in range(1 << n):
        candidates += 1
        weight = value = 0
        ids = []
        for i, (package_id, w, v) in enumerate(packages):
            if mask & (1 << i):
                weight += w
                value += v
                ids.append(package_id)
        if weight <= capacity:
            feasible += 1
```

```
            if value > best_value:
                best_value, best_weight, best_ids = value, weight, ids
    return best_ids, best_weight, best_value, candidates, feasible


packages = [("P1",2,6), ("P2",3,8), ("P3",4,9), ("P4",5,10)]
print(exhaustive_airlift(packages, capacity=7))
```

6.3 Independent correctness check

For four packages, there are $2^4 = 16$ subsets. At capacity 7, P2 + P3 weighs 7 and has value 17. P1 + P4 weighs 7 and has value 16; P1 + P3 has value 15. Any combination involving P2 + P4 or P3 + P4 exceeds capacity. Therefore P2 + P3 is the optimum.

Combination	Weight	Value	Feasible?
P1 + P2	5	14	Yes
P1 + P3	6	15	Yes
P1 + P4	7	16	Yes
P2 + P3	7	17	Yes – best
P2 + P4	8	18	No
P3 + P4	9	19	No
P1 + P2 + P3	9	23	No
P1 + P2 + P4	10	24	No
P1 + P3 + P4	11	25	No
P2 + P3 + P4	12	27	No
All four	14	33	No

6.4 Growth prediction and experiment sizes

The number of candidate subsets is 2^n. Adding one package doubles the search space; adding five packages multiplies it by 32.

Packages n	Candidate subsets 2^n	Interpretation
5	32	Small
10	1,024	Manageable
15	32,768	Noticeably more work
20	1,048,576	Over one million
25	33,554,432	Tens of millions
30	1,073,741,824	Over one billion

```
from time import perf_counter

def count_subsets(n):
    start = perf_counter()
    count = 0
    for mask in range(1 << n):
        count += 1
    return count, perf_counter() - start

for n in (5, 10, 15, 20):
    candidates, seconds = count_subsets(n)
    print("n =", n, "subsets =", candidates,
          "time =", round(seconds, 6), "seconds")
```

6.5 Deployment decision

Set a target of at most 1 second for a loading decision on the test machine. Run n = 5, 10, 15, and 20 and record the largest size that meets the target. This boundary must be measured locally because hardware and per-subset processing differ. For larger instances, consider dynamic programming, branch-and-bound, or approximation methods, each with its own trade-offs.

Q7 – Diagnose, Redesign and Validate

Chosen solution: Brute-Force Closest Pair

The closest-pair algorithm from Q4 is selected because its operation count grows quadratically. The redesign objective is to reduce pair-distance calculations while returning the same closest distance.

7A. Diagnose the computational bottleneck

The nested loops examine every unordered pair. The exact count is $n(n-1)/2$: 124,750 calculations for $n=500$ and 499,500 for $n=1,000$. Doubling n approximately quadruples the work. The bottleneck is the repeated evaluation of candidate pairs that spatial information could sometimes rule out.

7B. Define the design objective

Objective: preserve the exact closest-pair answer while reducing distance calculations on spatially distributed data. Measure the number of distance calculations, verify the returned minimum distance, and compare both methods on the same inputs.

7C. Revised computational strategy

The revised strategy sorts points by x-coordinate and stops comparing a point with later points once the x-coordinate difference is at least the best distance found. Since Euclidean distance cannot be smaller than the absolute x-coordinate difference, such later points cannot improve the current answer. Sorting costs $O(n \log n)$ and the scan can still be $O(n^2)$ in the worst case, but it may prune many comparisons in suitable data.

```
from math import sqrt, isclose

def closest_pair_brute(points):
    best_sq = float("inf")
    best_pair = None
    checks = 0
    for i in range(len(points) - 1):
        for j in range(i + 1, len(points)):
            checks += 1
            dx = points[i][0] - points[j][0]
            dy = points[i][1] - points[j][1]
            d2 = dx*dx + dy*dy
            if d2 < best_sq:
                best_sq, best_pair = d2, (points[i], points[j])
    return best_pair, sqrt(best_sq), checks

def closest_pair_sorted_prune(points):
    ordered = sorted(points, key=lambda p: p[0])
    best_sq = float("inf")
    best_pair = None
    checks = 0
    for i in range(len(ordered) - 1):
        j = i + 1
        while j < len(ordered):
            dx = ordered[j][0] - ordered[i][0]
            if dx * dx >= best_sq:
                break
            checks += 1
```

```
            dy = ordered[j][1] - ordered[i][1]
            d2 = dx*dx + dy*dy
            if d2 < best_sq:
                best_sq, best_pair = d2, (ordered[i], ordered[j])
            j += 1
    return best_pair, sqrt(best_sq), checks


for n in (100, 500, 1000):
    pts = [(i * 37 % (2*n+9), i * 53 % (2*n+13)) for i in range(n)]
    b_pair, b_dist, b_checks = closest_pair_brute(pts)
    r_pair, r_dist, r_checks = closest_pair_sorted_prune(pts)
    print("n =", n, "brute checks =", b_checks,
          "redesigned checks =", r_checks,
          "same distance =", isclose(b_dist, r_dist, rel_tol=1e-9))
```

7D. Prediction before implementation

Prediction: sorting adds preprocessing, but the pruning condition should reduce distance calculations for many ordinary point distributions. The primary performance measure is distance calculations; execution time is secondary.

7E. Implement and validate

Run both methods on n=100, 500, and 1,000 using identical point sets. The brute-force count must be n(n−1)/2. Copy the redesigned counts and local timings from the output. Confirm the distances match. If a test produces few or no reductions, report that result honestly; the method's worst case remains quadratic.

7F. Defend the final design

The redesign exploits x-coordinate ordering to avoid some comparisons. Its success is supported only if the measured distance calculations fall while the answer remains the same. The trade-offs are sorting time, extra memory for the sorted list, and a quadratic worst case when pruning is weak. Brute force remains suitable for tiny inputs and as a simple correctness reference. For larger deployments, a fully developed divide-and-conquer closest-pair algorithm or spatial index would be a stronger next step.

Appendix A – Raw Experimental Data and Reproduction

Run the code blocks to obtain machine-specific timings. Operation counts are the primary evidence; timing is secondary and depends on hardware, Python version, and background workload.

A1. Core formula reference

Algorithm / problem	Primary count or complexity	Important condition
Selection Sort	$n(n-1)/2$ comparisons	Input order does not change comparison count.
Optimised Bubble Sort	$O(n)$ best; $O(n^2)$ worst	Early stop requires a no-swap pass.
Insertion Sort	$O(n)$ best; $O(n^2)$ worst	Efficient when few inversions exist.
Sequential Search	1 to n comparisons	Missing target needs n comparisons.
Brute-force string matching	$O((n-m+1)m)$ worst case	Repeated prefixes cause repeated comparisons.
Closest Pair brute force	$n(n-1)/2$ distance calculations	Every unordered pair is checked.
Brute-force Convex Hull	$O(n^3)$ side tests	$O(n^2)$ candidate edges, each checks $O(n)$ points.
Exhaustive package search	2^n subsets	Each additional package doubles candidate subsets.

A2. Suggested raw-data recording sheet

Question	Input condition	Operation count	Runtime	Correctness
Q1	Sorted, n=8	Fill from program	Local run	Yes / No
Q1	Reverse, n=8	Fill from program	Local run	Yes / No
Q1	Nearly sorted, n=8	Fill from program	Local run	Yes / No
Q2	n=8 first/middle/last/missing	1 / 4 / 8 / 8	Optional	Yes / No
Q3	Text-pattern tests A/B/C	Fill from program	Optional	Yes / No
Q4	n=10, 100, 500	45 / 4,950 / 124,750	Local run	Yes / No
Q5	Set A and Set B	Side tests depend on points	Optional	Yes / No
Q6	n=5, 10, 15, 20	32 / 1,024 / 32,768 / 1,048,576	Local run	Yes / No
Q7	n=100, 500, 1,000	Compare both methods	Local run	Yes / No

A3. Experimental good practice

- Use the same test input for the original and redesigned solution.
- Do not count printing statements as algorithm operations.
- Use operation counts as the main evidence and timing as secondary evidence.
- Repeat timing measurements and report an average or median.
- Check edge cases: empty and one-item lists, missing search targets, empty patterns, duplicate or collinear points, and zero-weight packages.
- State exactly what counts as a comparison, shift, swap, distance calculation, or candidate subset.

Conclusion

Brute-force techniques are easy to understand and verify, but suitability depends on input size, input structure, and request frequency. Insertion Sort performs well on nearly sorted data; Sequential Search is acceptable for small or infrequently searched lists; brute-force string matching can repeat prefix comparisons; closest-pair and convex-hull brute-force methods become expensive as point counts rise; and exhaustive search grows exponentially with package count.

The Q7 redesign shows how identifying the computational bottleneck leads to a measurable objective. Sorting points by x-coordinate and pruning comparisons can reduce work on suitable data, although worst-case limitations remain. Engineering decisions should be defended with operation counts, correctness checks, and workload assumptions rather than runtime measurements alone.

References

1. Anany Levitin, Introduction to the Design and Analysis of Algorithms, Pearson.
2. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, Introduction to Algorithms, MIT Press.
3. Course notes and practical materials for CSA06 – Design and Analysis of Algorithms, CO2.